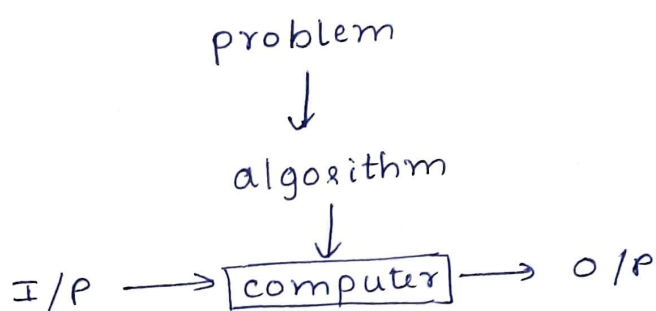


Algorithms

Algorithm is a sequence of unambiguous instructions for solving a problem for obtaining the desired O/P for any legitimate I/P in a finite amount of time.

- unambiguous.
- desired O/P
- legitimate I/P
- finite time



Properties:

- No ambiguity in the instructions.
- Range of I/P has to be specified carefully.
- Same algorithm can be expressed in different ways.
- Several algorithms for solving same problem might exist.
- Algorithms for a problem can be based on different ideas and can solve the problem with dramatically different speeds.

Algorithm to find GCD of two numbers:

20, 30

$$30 = 20 \times 1 + 10$$

$$20 = \boxed{10} \times 2 + 0$$

Find GCD (123, 36)

$$123 = 36 \times 3 + 15$$

$$36 = 15 \times 2 + 6$$

$$15 = 6 \times 2 + 3$$

$$6 = \boxed{3} \times 2 + 0$$

Euclid's Algorithm for computing $\gcd(m, n)$

Step 1: If $n = 0$, return the value of m as answer and stop; otherwise proceed to step 2.

Step 2: Divide m by n and assign the value of remainder to r .

Step 3: Assign the value of n to m and r to n .
Go to step 1.

Pseudocode:

Euclid(m, n):

// Input: two non-negative, not both zero integers
 m & n

// output: Greatest common divisor of m & n

while $n \neq 0$

$r = m \bmod n$

$m = n$

$n = r$

return m

Example:

$$m = 30 \quad n = 35$$

i) $r = 30 \% 35 = 30$

$$m = 35$$

$$n = 30$$

ii) $r = 35 \% 30 = 5 \quad [\because 30 \neq 0]$

$$m = 30$$

$$n = 5$$

iii) $r = 30 \% 5 = 0 \quad [\because 5 \neq 0]$

$$\boxed{m = 5}$$

$$n = 0$$

Consecutive Integer Checking Algorithm

Step 1: Assign the value of $\min\{m, n\}$ to t .

Step 2: Divide m by t . If the remainder of this division is zero, Go to step 3. Otherwise go to step 4.

Step 3: Divide n by t . If the remainder of this division is zero, return the value of t as the answer and stop. Otherwise go to step 4.

Step 4: Decrement t by 1 and Go to step 2.

Find $\text{GCD}(4, 6)$:

$$1) \ t = \min(4, 6) = 4$$

$$i) \quad \frac{4}{4} = 4 \div 4 = 0$$

$$6 \div 4 = 2 \neq 0$$

$$t = t - 1 = 3$$

$$ii) \quad \frac{4}{3} \quad 4 \div 3 = 1 \neq 0$$

$$t = t - 1 = 2$$

$$iii) \quad 4 \div 2 = 0$$

$$6 \div 2 = 0$$

$$\text{Hence } \text{GCD}(4, 6) = \underline{\underline{2}}.$$

Middle School Procedure:

Step 1: Find the prime factors of m .

Step 2: Find the prime factors of n .

Step 3: Identify all the common factors in the two prime expansions found in Step 1 and Step 2.

Step 4: Compute the product of all the common factors and return it as GCD of the numbers given.

Compute GCD (60, 24):

$$\begin{array}{r} 2 \overline{)60} \\ 2 \overline{)30} \\ 3 \overline{)15} \\ 5 \overline{)5} \\ 1 \end{array}$$

$$60 = 2 \times 2 \times 3 \times 5$$
$$24 = 2 \times 2 \times 2 \times 3$$

$$\text{GCD}(60, 24) = 2 \times 2 \times 3$$
$$= \underline{\underline{12}}$$

$$\begin{array}{r} 2 \overline{)24} \\ 2 \overline{)12} \\ 2 \overline{)6} \\ 3 \overline{)3} \\ 1 \end{array}$$

Sieve of Eratosthenes:

2	3	4	5	6	7	8	9	10	11
12	13	14	15	16	17	18	19	20	21
22	23	24	25	26	27	28	29	30	31
32	33	34	35	36	37	38	39	40	41
42	43	44	45	46	47	48	49	50	

2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37
41, 43, 47

Algorithm Sieve(n):

// Input : An integer $n \geq 2$

// Output : Array L of all prime numbers less than or equal to n.

for $p \leftarrow 2$ to n do

$A[p] \leftarrow p$

for $p \leftarrow 2$ to $\lfloor \sqrt{n} \rfloor$ do

if $A[p] \neq 0$

$j \leftarrow p * p$

while $j \leq n$ do

$A[j] \leftarrow 0$
 $j \leftarrow j+p$

$i \leftarrow 0$

for $p \leftarrow 2$ to n do

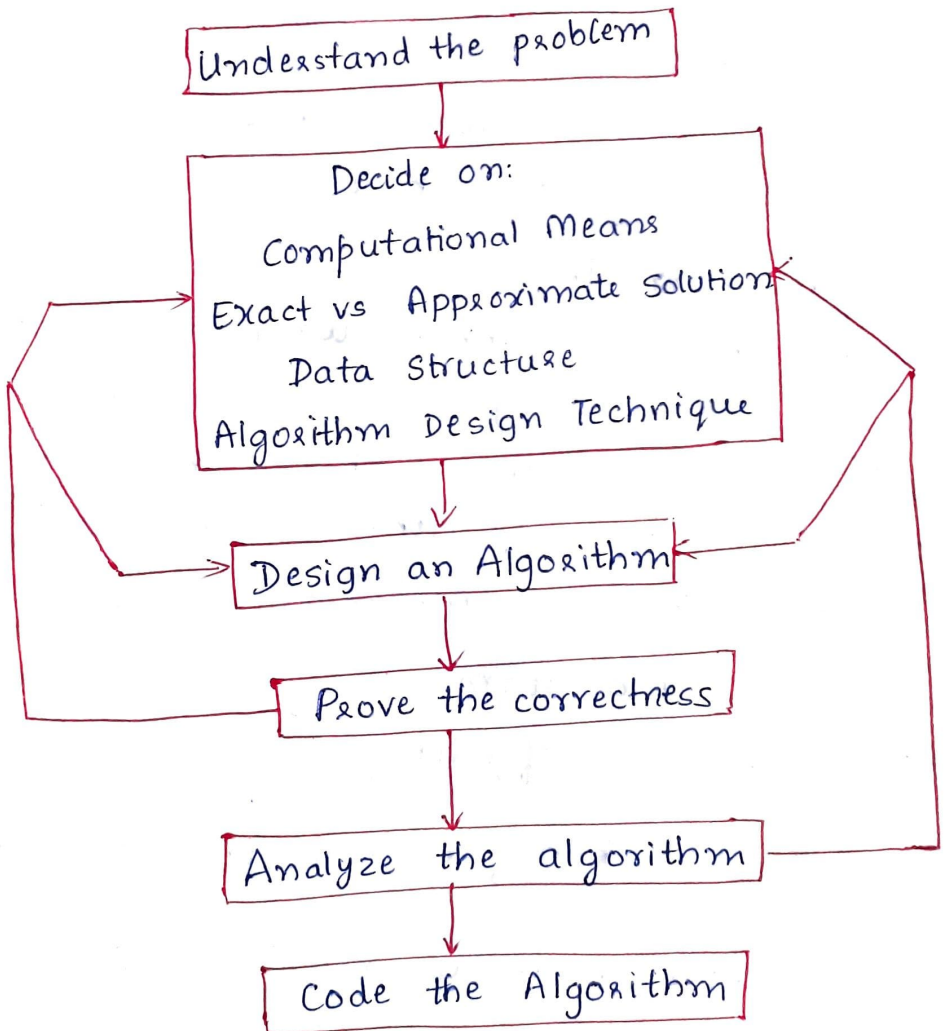
if $A[p] \neq 0$

$L[i] \leftarrow A[p]$

$i \leftarrow i+1$

return L

Fundamentals of Algorithm Problem Solving:



Understand the problem:

- ask doubts to clarify
- clearly define range of I/P
- An algorithm must work correctly for all inputs and not just most of the I/Ps.

Decision:

• Computational means $\begin{cases} \text{sequential algorithms} \\ \text{parallel algorithms} \end{cases}$

• Exact vs Approximate solutions.

↓
searching, sorting $\rightarrow$ square roots, integrals

• Data structure - Appropriate Data Structure

Algorithms + Data Structure = Program

• Algorithm Design Technique.

- Brute Force

- Divide and Conquer

- Decrease and Conquer

- Transform and conquer

Design an Algorithm:

- In terms of natural language, statements, pseudocode or flowchart.

Prove the correctness:

Proving the algorithm's wrongness can be done just by using one invalid input but proving the correctness should be done to show that the algorithm works for all inputs in decided range. (Mathematical Induction)

Analyze the Algorithm:

$\rightarrow$ In terms of efficiency $\begin{cases} \text{Time} \\ \text{Space} \end{cases}$

Time - time taken for algorithm to complete execution

Space - extra memory required for running algorithm.

Code the algorithm:

Develop the code for the algorithm using any specified programming language.

Important Problem Types:

- Sorting
 - stable
 - in place
- Searching
- String Problems
- Graph Problems
- Combinatorial problems
- Geometrical problems
- Numerical Problems

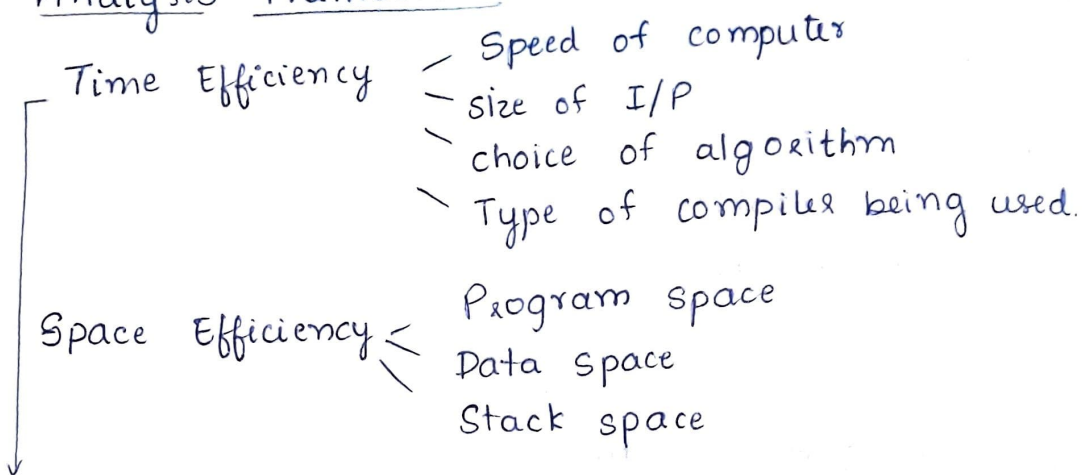
Fundamental Data Structures:

- Linear Data Structure
 - Arrays
 - Linked Lists
 - Stack
 - Queue
- Graphs
 - Directed and undirected graph.
 - Paths and cycles
 - Connectivity
 - Representation
 - adjacency matrix
 - adjacency list
 - weighted matrix
- Trees
 - Rooted Trees
 - Free Trees
 - Forest
 - Ordered Trees. — Binary Search Trees
- Sets & Dictionaries.
 - multiset or bag
 - key value pairs.

Abstract Data Type (ADT). — its an abstraction of data items with associated operations.

FUNDAMENTALS OF THE ANALYSIS OF ALGORITHM EFFICIENCY

Analysis Framework:



Measuring of I/P Size:

Searching, Sorting $\rightarrow$ 'n'

Polynomial Function $\rightarrow$ 'highest order'

Matrix $\rightarrow$ 'No. of elements'

String $\rightarrow$ string length or word count.

Numbers $\rightarrow$ No. of bits or digits in number

Units of Measuring Running Time:

- Basic operation - most Important operation in the algorithm.

Searching algorithm - 'comparison'

Matrix multiplication - 'multiplication'

Let C_{op} - time for execution of basic operation on some computer.

$C(n)$ - no. of times basic operation is executed

$$T(n) \approx C_{op} \cdot C(n)$$

$$C(n) = \frac{1}{2} n(n-1)$$

$$= \frac{1}{2} n^2 - \frac{1}{2} n$$

$$\approx \frac{1}{2} n^2$$

$$T(n) \approx c_{op} \cdot \frac{1}{2} n^2$$

$$\frac{T(2n)}{T(n)} \approx \frac{c_{op} \times \frac{1}{2} (2n)^2}{c_{op} \times \frac{1}{2} (n^2)}$$

$$\approx \frac{4n^2}{n^2}$$

$$\approx 4$$

Orders of Growth:

$$1 < \log_2 n < n < n \log_2 n < n^2 < n^3 < 2^n < n!$$

Sequential Search (A[0...n-1], k)

i ← 0

while i < n and A[i] ≠ k do

i ← i + 1

if i < n return i

else return -1

Best Case:

$$C_{best}(n) = 1$$

Worst Case:

$$C_{worst}(n) = n$$

Average case:

$$\begin{aligned} C_{avg}(n) &= \left[1 \cdot \frac{p}{n} + 2 \cdot \frac{p}{n} + \dots + i \cdot \frac{p}{n} + n \cdot \frac{p}{n} \right] + n(1-p) \\ &= \frac{p}{n} [1 + 2 + 3 + \dots + n] + n(1-p) \\ &= \frac{p}{n} \times \frac{n(n+1)}{2} + n(1-p) \\ &= \frac{p(n+1)}{2} + n(1-p) \end{aligned}$$

If $p = 1$, then $C_{avg}(n) = \frac{n+1}{2}$

If $p=0$, then $C_{avg}(n) = n$.

Order of growth:

n (Input size)	n	$\log_2 n$	$n \log_2 n$	n^2	n^3	2^n	$n!$
10	10	3.3	33	10^2	10^3	2^{10}	10!
10^2	10^2	6.6	660	10^4	10^6	2^{100}	$100!$
10^3	10^3	9.9	9900				

Slowest growing (above) - $\log_2 n$

Order: logarithmic linearithmic cubic Factorial

$$1 < \log_2 n < n < n \log_2 n < n^2 < n^3 < 2^n < n!$$

Constant linear quadratic Exponential

Asymptotic Notations:

Informal:

$O(g(n))$ is set of all function with smaller or same order of growth as $g(n)$.

$$n \in O(n^2)$$

$$n^3 \notin O(g(n))$$

$$\frac{n^2}{2} \in O(n^2)$$

$\Omega(g(n))$ is set of all functions that have larger or same order of growth as $g(n)$

$$n^3 \in \Omega(n^2)$$

$$n \notin \Omega(n^2)$$

$$\frac{1}{2}n^2 - n \in \Omega(n^2)$$

$\Theta(g(n))$ is a set of all functions that have the same order of growth as $g(n)$

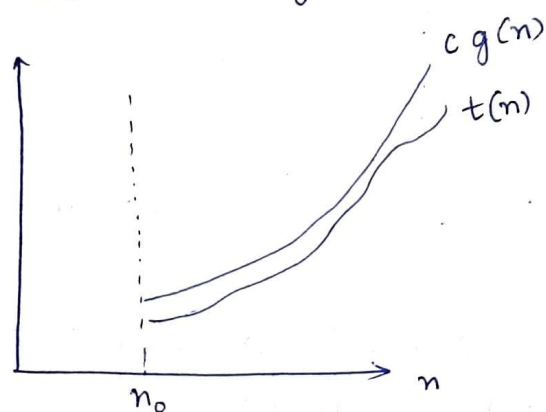
$$an^2 + bn + c \in \Theta(n^2)$$

$$n^2 + \log(n) \in \Theta(n^2)$$

O - Notation:

A function $t(n)$ is said to be in $O(g(n))$, denoted by $t(n) \in O(g(n))$ if $t(n)$ is bounded ^{above} by some constant multiple of $g(n)$ for all large n , i.e. there exists some positive constant c and some non-negative integer n_0 such that

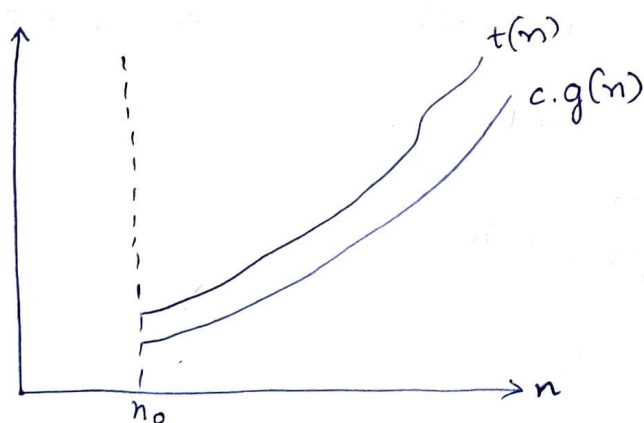
$$t(n) \leq c g(n) \text{ for all } n \geq n_0$$



Ω - Notation:

A function $t(n)$ is said to be in $\Omega(g(n))$ denoted by $t(n) \in \Omega(g(n))$, if $t(n)$ is bounded below by some positive constant multiple of $g(n)$ for all large n , i.e. if there exists some positive constant c and some non-negative integer n_0 such that,

$$t(n) \geq c g(n) \text{ for all } n \geq n_0$$

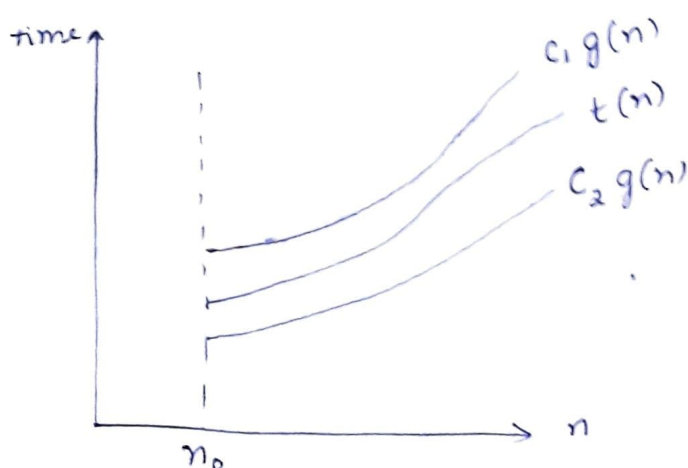


Θ Notation:

A function $t(n)$ is said to be in $\Theta(g(n))$ denoted by $t(n) \in \Theta(g(n))$, if $t(n)$ is bounded both above and below by some positive constant multiple of $g(n)$ for all large n , i.e. if these

exists same positive constant c_1 and c_2 and some non-negative integer n_0 such that,

$$c_2 g(n) \leq t(n) \leq c_1 g(n) \quad \text{for all } n \geq n_0$$



Problems:

- 1) Express $f(n) = 100n + 5$ in terms of Big Oh, Big Omega and Big Theta.

Big Oh:

$$f(n) = 100n + 5$$

$$f(n) \leq c \cdot g(n) \quad , \quad \forall n \geq n_0$$

$$\begin{aligned} c \cdot g(n) &= 100n + n \\ &= 101n \end{aligned}$$

$$f(n) \leq c \cdot g(n)$$

$$100n + 5 \leq 101n \quad \forall n \geq 5$$

$$c = 101$$

$$n_0 = 5$$

$$g(n) = n$$

Big Omega:

$$f(n) = 100n + 5$$

$$f(n) \geq c \cdot g(n)$$

$$100n + 5 \geq 100n \quad n \geq 0$$

$$c = 100$$

$$n_0 = 0$$

$$g(n) = n$$

Big Theta:

$$c_2 g(n) \leq f(n) \leq c_1 g(n) \quad \forall n \geq n_0$$

$$100n \leq 100n + 5 \leq 101n, \quad n \geq 5$$

$$c_2 = 100$$

$$c_1 = 101$$

$$g(n) = n$$

$$n_0 = 5$$

$$2) f(n) = 6 \cdot 2^n + n^2$$

Big Oh:

$$f(n) \leq c \cdot g(n)$$

$$6 \cdot 2^n + n^2 \leq 7 \cdot 2^n, \quad n \geq 0$$

$$n_0 = 0, \quad c = 7, \quad g(n) = 2^n$$

Big Omega:

$$f(n) \geq c \cdot g(n)$$

$$6 \cdot 2^n + n^2 \geq 6 \cdot 2^n, \quad n \geq 0$$

$$n_0 = 0, \quad g(n) = 2^n, \quad c = 6$$

Big Theta:

$$c_2 g(n) \leq f(n) \leq c_1 g(n)$$

$$6 \cdot 2^n \leq 6 \cdot 2^n + n^2 \leq 7 \cdot 2^n, \quad n \geq 0$$

$$c_2 = 6, \quad c_1 = 7, \quad g(n) = 2^n, \quad n_0 = 0$$

HW:

$$f(n) = 10n^3 + 5$$

$$\text{Oh: } c = 11, \quad g(n) = n^3, \quad n_0 = 2$$

$$\text{omega: } c = 10, \quad g(n) = n^3, \quad n_0 = 0$$

$$\text{Theta: } c_2 = 10, \quad c_1 = 11, \quad g(n) = n^3, \quad n \geq 2$$
$$n_0 = 2$$

* Useful property involving asymptotic notation:

Theorem:

$$\text{If } f_1(n) \in O(g_1(n))$$

$$\text{and } f_2(n) \in O(g_2(n))$$

$$\text{Then } f_1(n) + f_2(n) \in O(\max\{g_1(n), g_2(n)\})$$

Proof:

Let a_1, b_1, a_2 and b_2 be four arbitrary real numbers such that $a_1 \leq b_1$ and $a_2 \leq b_2$

$$a_1 + a_2 \leq b_1 + b_2$$

$$a_1 + a_2 \leq 2 \max\{b_1, b_2\}$$

$$f_1(n) \leq c_1 g_1(n) \quad \text{since } f_1(n) \in O(g_1(n))$$

$$\forall n \geq n_1$$

$$\text{Since } f_2(n) \in O(g_2(n)),$$

$$f_2(n) \leq c_2 g_2(n) \quad \forall n \geq n_2$$

Let us denote $c_3 = \max\{c_1, c_2\}$ and consider $n \geq \max\{n_1, n_2\}$ so that we can compare both inequalities. Adding both inequalities,

$$f_1(n) + f_2(n) \leq c_1 g_1(n) + c_2 g_2(n)$$

$$\leq c_3 g_1(n) + c_3 g_2(n)$$

$$\leq c_3 (g_1(n) + g_2(n))$$

$$\leq 2c_3 \max\{g_1(n), g_2(n)\}$$

$$\Rightarrow f_1(n) + f_2(n) \in O(\max\{g_1(n), g_2(n)\})$$

$$\text{where } c = 2c_3 = 2 \max\{c_1, c_2\}$$

$$\text{and } n \geq \max\{n_1, n_2\}$$

$\Rightarrow$ Hence the proof.

Best case - Ω

Average case - Θ

Worst case - O

Basic Efficiency classes:

1 - Constant, n - linear, $\log_2 n$ - logarithmic
 $n \log_2 n$ - linearithmic, n^2 - Quadratic, n^3 - Cubic,
 2^n - Exponential, $n!$ - Factorial

Mathematical Analysis of Non-Recursive Algorithms

ALGORITHM MaxElement($A[0 \dots n-1]$)

// Input: Array of n elements

// Output: The value of largest element in the array.

maxval $\leftarrow A[0]$

for $i \leftarrow 1$ to $n-1$ do

if $A[i] > \text{maxval}$

maxval $\leftarrow A[i]$

return maxval

→ Measure of I/P size - n

→ Basic operation - comparison

→ Best case, worst case and average case is same

$$C(n) = \sum_{i=1}^{n-1} 1$$

$$= n-1 \in \Theta(n)$$

Plan for analysing Time Efficiency of Non-Recursive Algorithms:

1. Decide on parameters indicating I/P size.
2. Identify the algorithm's basic operation.
3. Check if the no. of times the basic operation is executed depends only on size of input. If it depends on additional property, best case, worst case and average case have to be investigated.
4. Set up the sum indicating the no. of times the basic operation is executed.

5. Use standard formulas of sum manipulation to established order of growth.

Sum manipulation formulae:

$$\sum_{i=l}^u c a_i = c \sum_{i=l}^u a_i$$

$$\sum_{i=l}^u a_i \pm b_i = \sum_{i=l}^u a_i \pm \sum_{i=l}^u b_i$$

$$\sum_{i=l}^u 1 = u - l + 1 \quad \text{where } l \leq u.$$

$$\begin{aligned} \sum_{i=0}^n i &= 1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2} \\ &\approx \frac{n^2}{2} = \Theta(n^2) \end{aligned}$$

Element Uniqueness Problem:

ALGORITHM UniqueElements ($A[0 \dots n-1]$)

// Input : An array of n elements

// output: Returns true if all elements in A

// are distinct and false if all elements are not

// distinct

for $i \leftarrow 0$ to $n-2$ do

for $j \leftarrow i+1$ to $n-1$ do

if $A[i] = A[j]$

return false

return true

- Measure of I/P size : n
- Basic Operation : comparison
- Worst case

when last two elements
are only pair of equal
elements

when no elements are
equal

$$\begin{aligned}
C_{\text{worst}}(n) &= \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 \\
&= \sum_{i=0}^{n-2} [n-1 - (i+1) + 1] \\
&= \sum_{i=0}^{n-2} n - i - 1 \\
&= \sum_{i=0}^{n-2} n - \sum_{i=0}^{n-2} i - \sum_{i=0}^{n-2} 1 \\
&= n \sum_{i=0}^{n-2} 1 - \frac{(n-2)(n-1)}{2} - [n-2-0+1] \\
&= n[n-2-0+1] - \frac{(n-2)(n-1)}{2} - (n-1) \\
&= n(n-1) - (n-1) - \frac{(n-2)(n-1)}{2} \\
&= (n-1)^2 - \frac{(n-2)(n-1)}{2} \\
&= (n-1) \left[(n-1) - \frac{(n-2)}{2} \right] \\
&= (n-1) \left(\frac{2n-2-n+2}{2} \right) \\
&= \frac{n(n-1)}{2} \\
&\in O(n^2)
\end{aligned}$$

$$C_{\text{best}}(n) = 1 \in \Omega(1)$$

Matrix Multiplication:

$$\begin{array}{c} \text{row } i \end{array} \left[\begin{array}{c} \text{A} \\ \boxed{} \quad \boxed{} \quad \boxed{} \quad \boxed{} \quad \boxed{} \end{array} \right] \times \left[\begin{array}{c} \text{B} \\ \boxed{} \\ \boxed{} \\ \boxed{} \\ \boxed{} \\ \boxed{} \end{array} \right] = \left[\begin{array}{c} \text{C} \\ c[i,j] \end{array} \right]$$

col j

$$\begin{aligned}
c[i,j] &= A[i,0] * B[0,j] + A[i,1] * B[1,j] \\
&\quad + A[i,2] * B[2,j] + \dots + A[i,n-1] * B[n-1,j]
\end{aligned}$$

Algorithm Matrix Multiplication ($A[0..n-1], B[0..n-1]$):

// Input two n by n matrices

// Matrix Product $C = AB$

for $i \leftarrow 0$ to $n-1$ do

for $j \leftarrow 0$ to $n-1$ do

$c[i, j] \leftarrow 0.0$

for $k \leftarrow 0$ to $n-1$ do

$C[i, j] \leftarrow A[i, k] * B[k, j] + C[i, j]$

return C

- Size of Input - $n \times n$

- Basic Operation $C[i, j] \leftarrow C[i, j] + A[i, k] * B[k, j]$

Hence multiplication is basic.

No. of multiplications,

$$\begin{aligned}M(n) &= \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} \sum_{k=0}^{n-1} 1 \\&= \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} (n - i - 0 + i) \\&= \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} n \\&= \sum_{i=0}^{n-1} n \sum_{j=0}^{n-1} 1 \\&= \sum_{i=0}^{n-1} n (n - i - 0 + i) \\&= \sum_{i=0}^{n-1} n^2 \\&= n^2 \sum_{i=0}^{n-1} 1 \\&= n^2 (n - i - 0 + i) \\&= n^3\end{aligned}$$

$$\underline{\underline{f(n) \in \theta(n^3)}}$$

ALGORITHM Binary(n)

// Input: Positive decimal integer n

// output: The no. of binary digits in n's representation
// in binary

count $\leftarrow$ 1

while n > 1 do

count $\leftarrow$ count + 1

n $\leftarrow$ $\lfloor n/2 \rfloor$

return count

- Basic operation - while condition checking or
no. of additions
 $f(n) \in \Theta(\log_2 n)$

Mathematical Analysis of Recursive Algorithms:

Algorithm Factorial(n)

// computes n!

// I/P: non-negative integer n

// O/P: The value of n!

if n = 0 return 1

else return n * factorial(n-1)

$$F(n) = n * F(n-1), \quad n > 0$$

Let $m(n)$ be the no. of multiplications required
to compute $n!$. Then

$$m(n) = \begin{cases} 0 & n = 0 \\ 1 + m(n-1) & n > 0 \end{cases}$$

$\downarrow$ To multiply $n * F(n-1)$ $\hookrightarrow$ to compute $F(n-1)$

$$5! = 5 \times 4!$$

$$4! = 4 \times 3!$$

$$3! = 3 \times 2!$$

$$2! = 2 \times 1!$$

$$1! = 1 \times 0!$$

$$0! = 1$$

5 multiplications

$$\begin{aligned}
 M(n) &= 1 + M(n-1) \\
 &= 1 + (1 + M(n-2)) \\
 &= 2 + M(n-2) \\
 &= 3 + M(n-3) \\
 &\vdots \\
 &= i + M(n-i)
 \end{aligned}$$

$$\text{Let } n-i = 0 \Rightarrow i = n$$

$$\begin{aligned}
 M(n) &= n + M(n-n) \\
 &= n + M(0) \\
 &= n + 0
 \end{aligned}$$

$$\boxed{M(n) = n}$$

$$M(n) \in \Theta(n)$$

General Plan for Analyzing Time Efficiency of Recursive Algorithms:

- 1) Decide on parameter indicating input size.
- 2) Identify the algorithm's basic operation.
- 3) Check whether the no. of times basic operation is executed varies based on different inputs of same size; if so, worst case, best case and average case efficiency must be investigated separately.
- 4) Set up a recurrence relation, with an appropriate initial condition, for no. of times basic operation is executed.
- 5) Solve the recurrence relation or atleast ascertain order of growth of its solution.

Tower of Hanoi:

- Problem of moving n disks of different sizes using 3 rods from one rod to another such that, at a time only one disk is moved, and only upper disk can be moved and no disk can be

placed on smaller disk.



A
Source



B
Temp



C
Destination

Solution:

- 1) Recursively move $n-1$ disks from Source to Temp using Destination as auxiliary.
- 2) Move largest disk from source to destination rod.
- 3) Recursively move $n-1$ disks from Temp rod to destination rod using source rod as auxiliary.

Algorithm TowerOfHanoi(n, S, D, T)

if $n = 0$

return

if $n = 1$

print Move Disk 1 from S to D

return

TowerOfHanoi($n-1, S, T, D$)

print Move Disk n from S to D

TowerOfHanoi($n-1, T, D, S$)

Recurrence Relation:

$$M(n) = \begin{cases} 1 & n = 1 \\ 2M(n-1) + 1 & n > 1 \end{cases}$$

Solution by backward substitution:

$$M(n) = 2M(n-1) + 1$$

$$= 2[2M(n-2) + 1] + 1$$

$$= 2^2 M(n-2) + 2 + 1$$

$$= 2^3 M(n-3) + 2^2 + 2 + 1$$

$\vdots$

$$M(n) = 2^i M(n-i) + \underbrace{2^{i-1} + 2^{i-2} + \dots + 2^2 + 2 + 1}_{\text{In GP}}$$

$$\text{GP } n^{\text{th}} \text{ term sum} = \frac{a(r^n - 1)}{r - 1}$$

$$1 + 2 + 2^2 + \dots + 2^{i-1} = \frac{1(2^i - 1)}{2 - 1} = 2^i - 1$$

$$M(n) = 2^i M(n-i) + (2^i - 1)$$

$$\text{Substitute } n-i = 1 \Rightarrow i = n-1$$

$$M(n) = 2^{n-1} M(1) + 2^{n-1} - 1$$

$$= 2^{n-1} (1) + 2^{n-1} - 1$$

$$= 2^{n-1} \cdot 2 - 1$$

$$= 2^n - 1$$

$$\boxed{M(n) \in \Theta(2^n)}$$

Fibonacci Series:

0, 1, 1, 2, 3, 5, 8,

Simple recurrence Relation,

$$F(n) = F(n-1) + F(n-2), \quad n > 1$$

$$F(0) = 0, \quad F(1) = 1$$

Algorithm Fib(n)

// Input : non-negative number n

// o/p: The n^{th} Fibonacci number

if $n \leq 1$

return n

else return Fib(n-1) + Fib(n-2)

$$f(n) = \begin{cases} 0 & n = 0 \\ 1 & n = 1 \\ F(n-1) + F(n-2), & n > 1 \end{cases}$$

$$F(n) = F(n-1) + F(n-2)$$

Backward Substitution fails.

using solution for linear recurrence relations of second order,

$$a x(n) + b x(n-1) + c x(n-2) = 0$$

↓

$$a r^2 + b r + c = 0 \rightarrow \text{characteristic equation}$$

$$F(n) - F(n-1) - F(n-2) = 0$$

can be written as,

$$x^2 - x - 1 = 0$$

Roots of the above equation are:

$$x = \frac{+1 \pm \sqrt{(-1)^2 - 4 \times 1 \times -1}}{2(1)}$$

$$= \frac{1 \pm \sqrt{1+4}}{2} = \frac{1 \pm \sqrt{5}}{2}$$

$$x_1 = \frac{1 + \sqrt{5}}{2}, \quad x_2 = \frac{1 - \sqrt{5}}{2}$$

The solution can be represented in the form of,

$$F(n) = \alpha (x_1)^n + \beta (x_2)^n$$

$$F(n) = \alpha \left(\frac{1 + \sqrt{5}}{2} \right)^n + \beta \left(\frac{1 - \sqrt{5}}{2} \right)^n$$

To find value of α & β , we use base condition $F(0)$ & $F(1)$,

$$F(0) = 0$$

$$\alpha \left(\frac{1 + \sqrt{5}}{2} \right)^0 + \beta \left(\frac{1 - \sqrt{5}}{2} \right)^0 = 0$$

$$\alpha + \beta = 0 \Rightarrow \alpha = -\beta$$

$$F(1) = 1$$

$$\alpha \left(\frac{1 + \sqrt{5}}{2} \right) + \beta \left(\frac{1 - \sqrt{5}}{2} \right) = 1$$

$$\alpha \left(\frac{1 + \sqrt{5}}{2} \right) - \alpha \left(\frac{1 - \sqrt{5}}{2} \right) = 1$$

$$\frac{\cancel{2}\sqrt{5}}{2} \alpha = 1 \Rightarrow \alpha = \frac{1}{\sqrt{5}}$$

$$\therefore \beta = -\alpha = -\frac{1}{\sqrt{5}}$$

$$F(n) = \frac{1}{\sqrt{5}} \left(\frac{1 + \sqrt{5}}{2} \right)^n - \frac{1}{\sqrt{5}} \left(\frac{1 - \sqrt{5}}{2} \right)^n$$

$$= \frac{1}{\sqrt{5}} \left(\phi^n - \hat{\phi}^n \right)$$

where $\phi = \frac{1 + \sqrt{5}}{2}$, $\hat{\phi} = \frac{1 - \sqrt{5}}{2}$

$$\therefore \boxed{F(n) \in \theta(\phi^n)}$$

Fibonacci Addition Recurrence Relation:

$$A(n) = \begin{cases} 0 & n = 0 \\ 0 & n = 1 \\ A(n-1) + A(n-2) + 1 & n > 1 \end{cases}$$

$$A(n) = A(n-1) + A(n-2) + 1$$

$$A(n) - A(n-1) - A(n-2) = 1$$

$\Downarrow$

$$[A(n) + 1] - [A(n-1) + 1] - [A(n-2) + 1] = 0$$

$$\text{Let } B(n) = A(n) + 1$$

$$B(n-1) = A(n-1) + 1$$

$$B(n-2) = A(n-2) + 1$$

$$B(n) - B(n-1) - B(n-2) = 0$$

$$B(0) = 1$$

$$B(1) = 1$$

$$B(n) = F(n+1)$$

$$\Rightarrow A(n) = B(n) - 1$$

$$A(n) = F(n+1) - 1$$

$$= \frac{1}{\sqrt{5}} (\phi^{n+1} - \hat{\phi}^{n+1}) - 1$$

$$\Rightarrow A(n) \in \Theta(\phi^n)$$

Algorithm BinRec(n)

//I/P: Positive Decimal Integer n

//O/P: Number of digits in binary representation of n.

if $n = 1$

return 1

else return $1 + \text{BinRec}(\lfloor n/2 \rfloor)$

Recurrence Relation:

$$F(n) = \begin{cases} 1 & n = 1 \\ 1 + F(n/2) & n > 1 \end{cases}$$

$$\begin{aligned} F(n) &= 1 + F(n/2) \\ &= 1 + (1 + F(n/4)) \\ &= 2 + F(n/2^2) \\ &= 2 + (F(n/2^3) + 1) \\ &= 3 + F(n/2^3) \\ &\vdots \\ &= i + F(n/2^i) \end{aligned}$$

$$\text{Let } \frac{n}{2^i} = 1 \Rightarrow n = 2^i \text{ or } i = \log_2 n$$

$$\begin{aligned}
 F(n) &= \log_2 n + F(1) \\
 &= \log_2 n + 0 = \log_2 n
 \end{aligned}$$

$$F(n) \in \Theta(\log_2 n)$$

Fibonacci - Iterative Fib(n):

$F[0] \leftarrow 0; \quad F[1] \leftarrow 1$

for $i \leftarrow 2$ to n do

$F[i] \leftarrow F[i-1] + F[i-2]$

return $F[n]$

Homework:

ALGORITHM Sum(n)

// Input: A non-negative integer n

$S \leftarrow 0$

for $i \leftarrow 1$ to n do

$S \leftarrow S + i$

return S

a) What does this algorithm compute?

b) What is the basic operation?

c) How many times is the basic operation executed?

d) What is the efficiency class of this algorithm?